

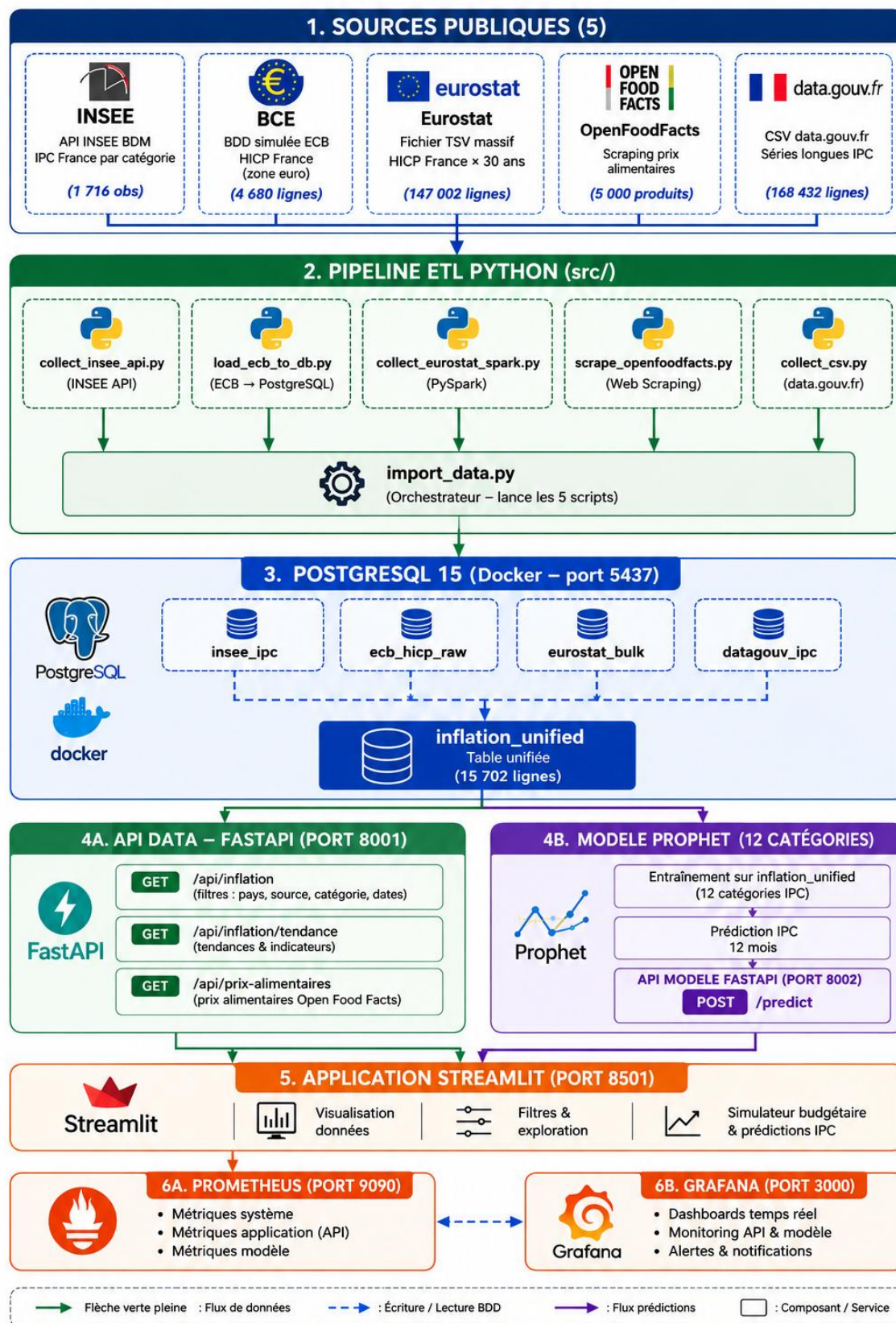
SPECIFICATIONS TECHNIQUES — INFLATION TRACKER (C15)

Projet : Inflation Tracker France — B3 RNCP Développeur IA

Auteur : Alberto Bongue | Date : 2026-07

Architecture générale

ARCHITECTURE GLOBALE – INFLATION TRACKER



Décision technique #1 : Normalisation des données (C3)

Problème : Garbage In, Garbage Out

Audit réalisé lors de l'intégration des sources : la colonne valeur de **inflation_unified** contenait des grandeurs incomparables selon la source.

Source	Valeur brute	Unité	Problème
INSEE	119.17	Indice base 100=2015	Reference
DATAGOUV	119.17 et 2.4	Mélange indices + taux %	Aucun filtre sur UNIT_MEASURE
ECB	2.4	Taux ANR (Annual Rate) %	Pas un indice
EUROSTAT	3.2	Taux RCH_A %	Pas un indice

Solution appliquée

Règle unique : inflation_unified.valeur = indice IPC base 100 = 2015 pour toutes les sources.

DATAGOUV (collect_csv.py)

```
---
286     # --- Filtre 2 : unité = indice (IX) uniquement ---
287     # UNIT_MEASURE peut être :
288     #   "IX" → indice IPC (ex: 119.17) ← ce qu'on veut
289     #   "RCH_A" → taux de variation annuel en % (ex: 2.4) ← à exclure
290     #   "RCH_M" → taux mensuel % ← à exclure
291     # Mélanger indices et taux dans la même colonne 'valeur' = non-sens
292     unites_disponibles = df["UNIT_MEASURE"].unique()
293     log.info(f"Unités disponibles : {unites_disponibles}")
294     df = df[df["UNIT_MEASURE"] == "IX"].copy()
295     log.info(f"Filtre UNIT MEASURE='IX' : {len(df)} lignes indices")

297     # --- Filtre 3 : indicateur = valeur d'indice (pas variation YoY) ---
298     # IND_TYPE distingue la valeur brute de l'indice de ses dérivés :
299     #   "IX" → valeur de l'indice (ex: 62.81) ← ce qu'on veut
300     #   "YOY" → variation année-sur-année, parfois stockée avec UNIT_MEASURE="IX"
301     #           (ex: 0.40 = variation de 0.40 point d'indice) – valeur parasite
302     # Sans ce filtre, des lignes avec valeur 0.40 se glissent dans les indices.
303     if "IND_TYPE" in df.columns:
304         types_ind = df["IND_TYPE"].unique()
305         log.info(f"IND_TYPE disponibles : {types_ind}")
306         df = df[df["IND_TYPE"] == "IX"].copy()
307         log.info(f"Filtre IND_TYPE='IX' : {len(df)} lignes valeurs d'indice")
308     else:
309         log.warning("Colonne IND TYPE absente – filtre YoY ignoré")

311     # --- Filtre 4 : France nationale uniquement (pas les DOM/COM) ---
312     # GEO code le territoire géographique :
313     #   "F" → France métropolitaine (national) ← ce qu'on veut
314     #   "973" → Guyane, "971" → Guadeloupe, etc.
315     # Sans ce filtre, la même catégorie COICOP pour le même mois
316     # génère 3 lignes ou plus (une par territoire), causant des doublons
317     # dans inflation_unified après l'agrégation.
318     if "GEO" in df.columns:
319         geos = df["GEO"].unique()
320         log.info(f"GEO disponibles ({len(geos)}) : {geos[:20]}")
321         df = df[df["GEO"] == "F"].copy()
322         log.info(f"Filtre GEO='F' : {len(df)} lignes France nationale")
323     else:
324         log.warning("Colonne GEO absente – filtre territoire ignoré")
```

ECB (load_ecb_to_db.py)

```
# Avant : ICP/M.{PAYS}.N.{COICOP}.4.ANR (taux de variation annuel  
%)  
# Apres : ICP/M.{PAYS}.N.{COICOP}.4.INX (indice HICP base  
2015=100)
```

EUROSTAT (collect_eurostat_spark.py)

```
# Avant : dataset prc_hicp_manr, UNIT_FILTRE = "RCH_A" (taux %)  
# Apres : dataset prc_hicp_midx, UNIT_FILTRE = "I15" (indice  
base 2015=100)  
# Attention : prc_hicp_midx utilise "I15", pas "INX_A_AVG"  
(moyennes annuelles)
```

Cas particulier : DATAGOUV rebase 2025

En 2025, l'INSEE a rebase toutes ses séries de 2015 vers 2025. Le fichier DATAGOUV ne contient plus aucune donnée `BASE_PER = "2015"`.

Conséquence : DATAGOUV (base 2025) et INSEE/ECB/EUROSTAT (base 2015) ne sont pas comparables en valeur absolue dans `inflation_unified`. Les tendances et variations relatives restent valides.

Décision : conserver DATAGOUV malgré l'incompatibilité - c'est la seule source couvrant la France depuis 1996. Toute interface utilisateur doit indiquer la base de référence de la source affichée.

Impact sur le modèle Prophet

Prophet est entraîné sur `source="INSEE"` (base 100=2015). Ses prédictions sont dans la même unité que toutes les sources après normalisation - overlay historique cohérent dans l'application, métriques comparables entre sources.

Décision technique #2 : API REST (C5/C9)

Deux APIs FastAPI séparées :

- Port 8001 : API data - expose `inflation_unified` (~15 700 lignes) + prix alimentaires
- Port 8002 : API modèle - expose les 12 modèles Prophet + métriques

Séparation justifiée : cycle de vie différent (données vs modèle), scalabilité indépendante, monitoring séparé (métriques Prometheus distinctes par API).

Décision technique #3 : Prophet vs LSTM vs ARIMA (C7/C8)

Voir `veille_C6_final.pdf` - Section 3 (tableau comparatif Prophet/ARIMA/LSTM sur 10 critères) et Sections 5.1-5.3 (justification du choix Prophet).

Résumé : Prophet retenu pour sa gestion native de la saisonnalité mensuelle, sa robustesse aux valeurs manquantes, et la lisibilité de ses composantes (tendance + saisonnalité) - critique pour un projet d'analyse économique.

Décision technique #4 : Split d'évaluation temporel strict

Train : Jan 2020 vers Déc. 2024 (60 mois)

Eval : Jan 2025 vers Déc. 2025 (12 mois, held-out)

Justification : un split temporel strict (pas de shuffle) est impératif pour les séries temporelles. Mélanger des observations futures dans le train introduit du data leakage et surestime les performances.

Décision technique #5 : Exclusion catégorie 12 du modèle Prophet (C8)

Catégorie concernée : 12 - Biens et services divers (soins personnels, protection sociale, assurances, services financiers divers).

Cause technique : model/train.py charge dynamiquement toutes les catégories présentes dans inflation_unified pour source="INSEE". La série **INSEE BDM** correspondant à la catégorie 12 n'a pas été intégrée dans collect_insee_api.py - la donnée est absente de la base.

Justification métier : la catégorie 12 regroupe des prix structurellement hétérogènes (coiffure, frais bancaires, assurances) dont les mécanismes de formation diffèrent fondamentalement des autres catégories. Son exclusion est une décision de qualité modèle, pas seulement une contrainte de données.

Périmètre du modèle : 12 catégories entraînées (COICOP 00-11), toutes avec n_train=60 et n_eval=12. MAE moyenne : 1.53 pts IPC.

Décision technique #6 : Réentraînement manuel (C13)

Le réentraînement des modèles Prophet n'est pas automatisé en CI/CD.

Raison : l'entraînement nécessite CmdStan (compilateur C++ ~500 Mo) et ~15 minutes de calcul pour les 12 catégories. Ce coût est incompatible avec un pipeline CI déclenché à chaque push.

Procédure de réentraînement :

```
python model/train.py          # genere les 12 .pkl + metrics.json
git add model/metrics.json     # versionne - trace les
performances
# les .pkl sont dans .gitignore (binaires lourds, régénérables)
```

Déclencheur : à chaque mise à jour des données INSEE dans inflation_unified (mensuelle) ou à chaque changement d'hyperparamètres Prophet.

Limites connues

Limite	Impact	Décision
JAVA_HOME non défini dans la session VS Code	PySpark échoue si Java n'est pas dans le PATH	Fermer/rouvrir le terminal après installation Java
OpenFoodFacts collecte mais non intégré en UI	Les prix alimentaires réels ne sont pas visualisés	Volume insuffisant et couverture partielle - backlog v2
DATAGOUV rebase 2025	Valeurs absolues incomparables avec base 2015	Tendances relatives valides - note affichée dans specs fonctionnelles
Catégorie 12 exclue du modèle	12 catégories au lieu de 13	Voir Décision technique #5

Stack technique

Composant	Technologie	Justification
Base de données	PostgreSQL 15 (Docker)	ACID, NUMERIC(10,4) sans erreur float
API	FastAPI + SQLAlchemy	Async, Pydantic validation, auto-docs
Modèle	Prophet (Meta)	Saisonnalité IPC mensuelle, robuste aux gaps
Application	Streamlit	Prototypage IA rapide, pas de JS requis
Monitoring	Prometheus + Grafana	Standard industrie, alertes MAE
CI/CD	GitHub Actions	Tests automatisés, skip intégration en CI
Big Data	PySpark (Eurostat — ~3,5M lignes brutes, 27 pays, filtrées à 147 002 lignes pour la France)	Preuve traitement distribue (C2)

Décision technique #7 : Monitoring applicatif (C20)

Contexte : le monitoring existant (C11) ne couvrant que le modèle Prophet (MAE, latence prédictions), laissant l'API data sans aucune visibilité aucune métrique ne permettait de détecter une panne ou une dégradation de l'API data indépendamment du modèle.

Solution appliquée : l'API data est instrumentée avec Prometheus, en miroir de C11 mais avec ses propres métriques dédiées :

- Compteurs de requêtes par endpoint (inflation_data_requests_total)
- Histogramme de latence (inflation_data_request_latency_seconds)
- Compteur d'erreurs par type (inflation_data_errors_total)

Deux alertes dédiées (APIDataIndisponible, TauxErreurs500APIData) surveillent la disponibilité et le taux d'erreurs 500 de l'API data, complétant les 4 alertes existantes sur l'API modèle.

Justification : séparer les métriques par API (data vs modèle) permet d'identifier immédiatement quelle API est en cause en cas d'incident, sans avoir à corréliser manuellement des métriques mélangées, cohérent avec la séparation des deux services décidés en #2.